

Documento 2

Arquitectura Técnica

Sistema Integral de Gestión Comercial para una Importadora Mayorista

Campo	Descripción
Versión	1.0 - Documento definitivo de arquitectura técnica
Fecha	Julio 2026
Estado	Fuente técnica oficial para el desarrollo del sistema
Uso	Guía de construcción para Flask, módulos, base de datos, templates, Git y despliegue
Alcance	Página pública tipo catálogo, panel administrativo, inventario en tienda, cotizaciones y ventas internas
Fuentes base	Documento 0 - Project Context; Documento 4 - Development Roadmap; Documento 6 - Architecture Decision Record
Restricción principal Este documento no cambia reglas de negocio, no modifica el Roadmap ni el ADR, y no propone tecnologías distintas a las aprobadas. Su función es organizar técnicamente lo que ya fue definido para convertirlo en una guía de desarrollo clara.	

Índice

Sección	Título	Página
1	Introducción	3
2	Objetivo del documento	3
3	Alcance	3
4	Arquitectura general del sistema	5
5	Explicación de Flask utilizando este proyecto como ejemplo	6
6	Explicación de Jinja2 Templates	7
7	Tecnologías utilizadas	8
8	Arquitectura modular	8
9	Estructura completa del proyecto	10
10	Explicación de cada archivo importante	10
11	Organización de Templates	11
12	Organización de Static	11
13	Organización de la Base de Datos	12
14	Git Flow del proyecto	13
15	Flujo de desarrollo	13
16	Convenciones de nombres	14
17	Buenas prácticas	14
18	Cumplimiento de los lineamientos del profesor	15
19	Diagramas de arquitectura	16
20	Diagramas de flujo	18
21	Preguntas para la presentación	20

Nota: el índice utiliza numeración interna del documento y paginación generada en la exportación final.

Fuentes oficiales utilizadas y criterios de interpretación

Este documento se construye a partir de los documentos base cargados en el proyecto. Cuando una decisión técnica o de negocio aparece en más de un documento, se conserva la versión coherente con el ADR y el Development Roadmap.

Fuente	Uso dentro de esta arquitectura
Documento 0 - Project Context	Define propósito, alcance, módulos principales, reglas de negocio por categoría, estados del sistema y restricciones de alcance.
Documento 4 - Development Roadmap	Define fases, sprints, ramas Git, Definition of Done, orden de construcción y flujo de trabajo por chats.
Documento 6 - ADR	Registra decisiones aprobadas: producto único, arquitectura modular, inventario descontado al pagar, no eliminación física de productos, diseño visual y metodología.
Criterio de coherencia Si en el futuro se cambia una regla que impacte arquitectura, UI, base de datos o roadmap, primero debe actualizarse el ADR desde el chat MASTER y después reflejarse en los documentos afectados.	

1. Introducción

El Sistema Integral de Gestión Comercial para una Importadora Mayorista es una aplicación web orientada a ordenar el flujo real de una empresa que vende principalmente cosméticos, juguetes y flores por volumen. La solución no se plantea como una tienda en línea con pagos automáticos, sino como una herramienta de consulta, cotización, venta interna e inventario en tienda.

La arquitectura técnica se diseña para que un mismo producto alimente el catálogo público, el inventario, las cotizaciones y las ventas. Esta decisión evita duplicidad de datos, reduce errores manuales y permite que la empresa conserve una trazabilidad clara de sus operaciones.

El sistema será desarrollado con Flask, plantillas Jinja2, HTML, Bootstrap, CSS, SQLite, Git, GitHub y Vercel. Estas tecnologías se mantienen dentro del alcance aprobado y permiten construir un proyecto académico-profesional, modular, explicable y desplegable.

- El catálogo público muestra productos y disponibilidad sin revelar cantidades exactas de stock.
- El panel administrativo gestiona productos, usuarios, inventario, cotizaciones, ventas e historial.
- El panel de vendedores permite buscar productos, crear pedidos, marcar pagos y generar comprobantes.
- El inventario se descuenta únicamente cuando el pedido queda marcado como pagado.
- Los productos no se eliminan físicamente; se activan, inactivan o archivan para conservar historial.

2. Objetivo del documento

El objetivo de este documento es definir la arquitectura técnica que guiará la construcción del sistema. La intención es que cualquier integrante del equipo pueda entender cómo se organiza la aplicación, qué módulos existen, qué carpetas y archivos son importantes, cómo fluye la información y cómo se debe trabajar con Git sin contradecir el Roadmap ni el ADR.

Este documento servirá como referencia para iniciar la Fase 2 del Roadmap, donde se crea la base Flask, se prepara el repositorio, se definen dependencias y se establece la estructura modular inicial. También servirá como guía para las fases posteriores: módulo público, panel administrativo, sistema de ventas, integración, pruebas y despliegue.

Objetivo técnico	Aplicación práctica
Definir la estructura Flask	Indica cómo separar rutas, servicios, modelos, templates y archivos estáticos.
Explicar las tecnologías	Justifica el uso de Python, Flask, HTML, Bootstrap, CSS, SQLite, Git, GitHub y Vercel.
Mantener coherencia	Alinea el desarrollo con las reglas aprobadas en el ADR y con el orden del Development Roadmap.
Facilitar presentación	Incluye diagramas, flujos y preguntas técnicas para defender el proyecto ante el profesor.

3. Alcance

El alcance técnico corresponde a la versión aprobada del proyecto. Se construye un sistema web con catálogo público, inventario en tienda y ventas internas. No se agregan módulos externos que no formen parte del flujo de consulta, cotización, pago en tienda y control de inventario.

Incluido en la versión	Descripción técnica
Página web pública tipo catálogo	Home, navegación, categorías, subcategorías, listado de productos, detalle y cotización.
Gestión de productos	Alta, edición, activación, inactivación, archivo, imagen, código único y presentación comercial.

Inventario en tienda	Existencias internas, bajo stock configurable por subcategoría, movimientos e historial.
Cotizaciones	Selección de productos, datos del cliente, número único y generación de documento visual.
Ventas internas	Pedido, cálculo de totales, pago en tienda, descuento de inventario y comprobante.
Usuarios internos	Administrador y vendedores con permisos diferenciados.
Estados	Producto, disponibilidad pública y pedido según lo definido en el proyecto.
No incluido	Motivo de exclusión
Inventario en bodega	Está fuera del alcance aprobado para evitar ampliar innecesariamente el sistema.
Facturación fiscal o SAT	No forma parte del objetivo de esta versión.
Pagos en línea	El flujo real aprobado es consulta, pedido, pago en tienda y confirmación interna.
Manejo de proveedores	No pertenece al flujo principal de venta en tienda.
Control de importaciones	Queda fuera de la versión para mantener el proyecto enfocado.
Módulos ajenos al flujo comercial definido	No se agregan funcionalidades que no estén en el Roadmap o ADR.

3.1 Reglas de negocio que condicionan la arquitectura

- Cosméticos: se venden por caja o display; no se modela venta por unidad suelta si la presentación comercial no lo permite.
- Juguetes: el mínimo de venta depende del precio del producto según la regla aprobada por el proyecto.
- Flores: se venden por ramos y el mínimo aprobado es de 3 ramos por código.
- El producto tiene código único definido por la empresa y no debe duplicarse para catálogo, inventario o ventas.
- Los cálculos de precios, subtotales y totales son automáticos dentro del flujo de cotización o venta.

3.2 Límites técnicos derivados del alcance

Decisión de alcance	Impacto técnico en la arquitectura
Sin pagos en línea	El sistema registra estado de pago confirmado en tienda, pero no integra pasarelas externas.
Sin facturación fiscal	Los documentos generados son cotización o comprobante interno, no factura fiscal.
Sin inventario de bodega	El modelo de inventario se enfoca en existencias de tienda y movimientos internos aprobados.
Sin proveedores ni importaciones	La base de datos no incorpora flujos de compra, embarque o recepción internacional.
Sin exposición de datos sensibles al público	Las rutas públicas reciben información filtrada y no muestran stock exacto ni datos internos.

4. Arquitectura general del sistema

La arquitectura general se organiza como una aplicación web Flask de renderizado del lado del servidor. El navegador solicita páginas, Flask procesa la petición mediante rutas organizadas en módulos, los servicios aplican reglas de negocio, los modelos se comunican con SQLite y las plantillas Jinja2 generan HTML con Bootstrap y CSS.

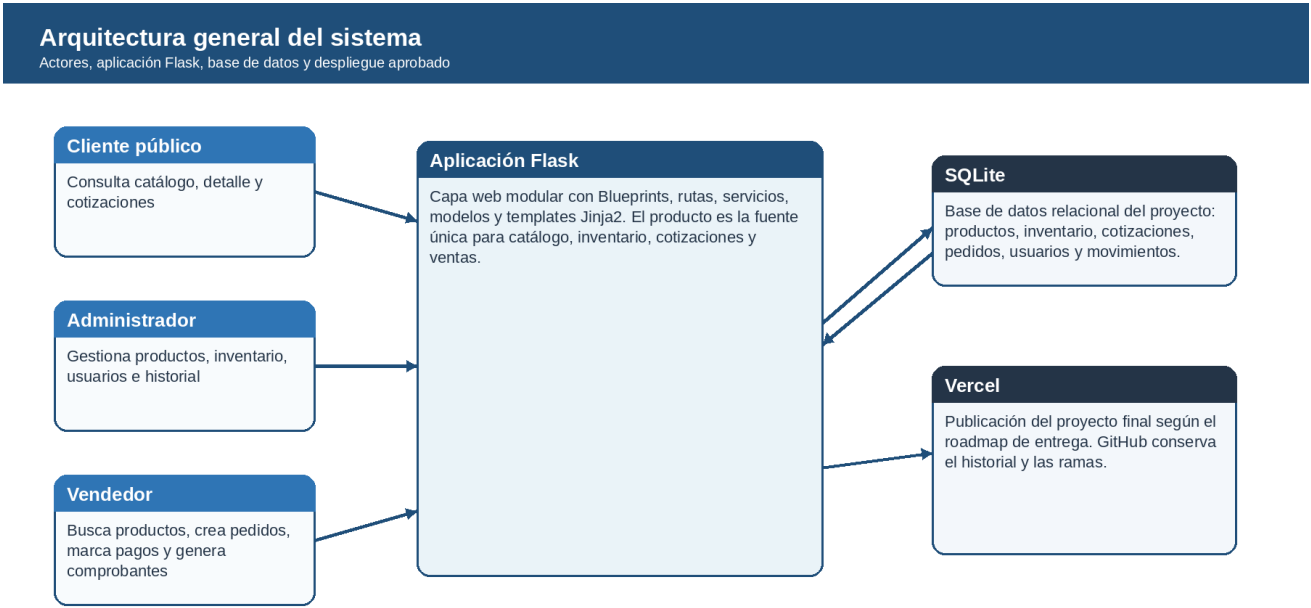


Figura 1. Arquitectura general del sistema.

4.1 Principios arquitectónicos

Principio	Aplicación en el sistema
Producto como fuente única	El producto central alimenta catálogo, inventario, cotizaciones y ventas, evitando duplicidad.
Arquitectura modular	Cada módulo se organiza con rutas, servicios, templates y recursos propios cuando corresponda.
Separación de responsabilidades	Las plantillas muestran información; los servicios concentran reglas de negocio; los modelos representan datos.
Trazabilidad	No se eliminan productos ni documentos relevantes; se usan estados e historial.
Seguridad por rol	El público no ve información sensible; el administrador y el vendedor tienen acciones distintas.
Desarrollo controlado	Cada cambio se trabaja en feature/*, se revisa por Pull Request y se integra a dev.

4.2 Actores del sistema

Actor	Responsabilidad	Acceso principal
Cliente público	Consulta catálogo, categorías, productos y genera cotización.	Módulo Público
Administrador	Gestiona productos, inventario, usuarios, ventas, cotizaciones e historial.	Panel Administrativo
Vendedor	Busca productos, crea pedidos, marca pagos y genera comprobantes.	Panel de Vendedores
Sistema	Calcula totales, controla estados y actualiza inventario al confirmar pago.	Servicios internos y base de datos

5. Explicación de Flask utilizando este proyecto como ejemplo

Flask es el framework web que recibirá las solicitudes del navegador, ejecutará la lógica necesaria y devolverá páginas HTML generadas desde templates. En este proyecto, Flask actúa como el centro de coordinación entre la interfaz, los módulos funcionales, los servicios y la base de datos SQLite.

Flujo de una petición Flask

Ejemplo de navegación desde el catálogo hasta la respuesta HTML

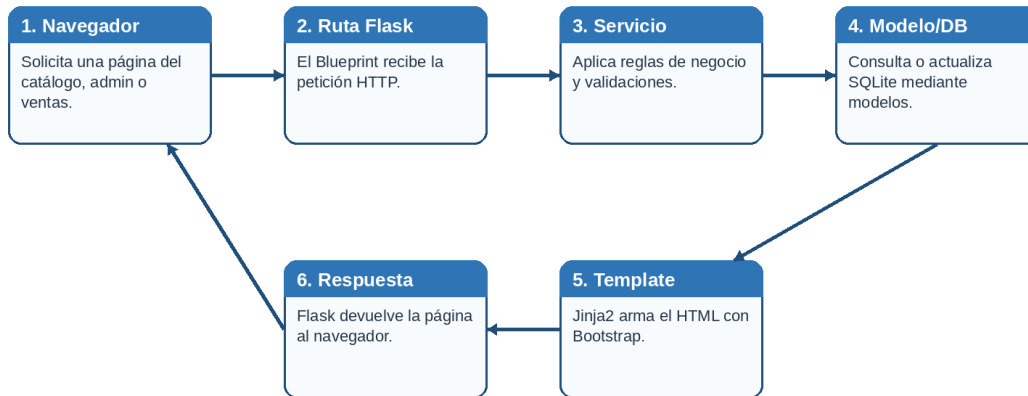


Figura 2. Flujo de una petición Flask aplicada al catálogo.

5.1 Ejemplo aplicado al catálogo público

1. El cliente entra al catálogo desde el navegador.
2. La ruta del módulo público recibe la solicitud.
3. El servicio del catálogo solicita productos activos, aplica filtros y evita exponer el stock exacto.
4. Los modelos obtienen la información desde SQLite.
5. Flask entrega los datos permitidos a un template Jinja2.
6. El template genera una página HTML con Bootstrap y CSS.
7. El navegador muestra una vista de catálogo profesional y responsive.

5.2 Uso de Blueprints

La arquitectura modular aprobada se implementa naturalmente con Blueprints de Flask. Un Blueprint permite separar rutas por área funcional sin mezclar el catálogo público con administración, ventas o cotizaciones. Esto facilita el trabajo por chats, ramas Git y sprints, porque cada módulo puede avanzar de forma controlada.

Blueprint o módulo técnico	Responsabilidad
public	Home, categorías, catálogo, detalle de producto y generación inicial de cotización.
admin	Dashboard, gestión de productos, inventario, usuarios, ventas y cotizaciones internas.
sales	Buscador de vendedor, creación de pedidos, pago y comprobantes.
quotes	Gestión de cotizaciones y posible conversión a pedido dentro del flujo aprobado.
auth	Ingreso y control de sesión para administrador y vendedores.
Importante El nombre exacto de cada carpeta puede ajustarse durante la Fase 2 siempre que conserve la arquitectura modular aprobada y no cambie el alcance funcional.	

6. Explicación de Jinja2 Templates

Jinja2 es el motor de plantillas que Flask utiliza para transformar datos del sistema en páginas HTML. En este proyecto, las plantillas permiten crear una interfaz consistente para el cliente público, el administrador y el vendedor sin repetir la estructura completa de cada página.

6.1 Herencia de templates

La aplicación debe contar con una plantilla base que contenga la estructura común: encabezado, navegación, carga de Bootstrap, carga de CSS propio, bloque de contenido, mensajes visuales y footer. Las pantallas específicas heredan esa estructura y reemplazan únicamente el contenido central.

Elemento de template	Uso dentro del proyecto
base.html	Estructura global compartida para las páginas.
partials	Fragmentos reutilizables como navbar, footer, tarjetas de producto o alertas.
templates públicos	Home, catálogo, detalle y cotización visible para clientes.
templates admin	Dashboard, formularios de producto, inventario, usuarios y listados internos.
templates ventas	Buscador, pedido, pago y comprobante.

6.2 Regla de separación entre template y lógica

Los templates no deben concentrar reglas de negocio complejas. Su función es mostrar información preparada por Flask y los servicios. Por ejemplo, el template puede mostrar una etiqueta de "Baja disponibilidad", pero la decisión de cuándo un producto entra en bajo stock debe calcularse antes, usando la regla aprobada por subcategoría.

Debe estar en templates	Debe estar en servicios/modelos
Estructura visual, tablas, tarjetas, formularios y mensajes.	Reglas de mínimos de venta, cálculos, validaciones y cambios de inventario.
Uso de clases Bootstrap y estilos CSS.	Consulta de productos activos, estados y permisos.
Presentación de datos permitidos al usuario.	Protección de stock exacto y datos sensibles internos.

7. Tecnologías utilizadas

Las tecnologías se seleccionan por compatibilidad con el alcance, facilidad de explicación, desarrollo modular y alineación con el Roadmap. No se agregan tecnologías diferentes a las aprobadas para el proyecto.

Tecnología	Rol	Justificación
Python	Lenguaje principal del backend.	Permite construir lógica clara, legible y fácil de explicar. Es adecuado para un proyecto Flask modular.
Flask	Framework web.	Es ligero, flexible y compatible con Blueprints para separar módulos como público, administración y ventas.
HTML / Templates	Estructura de las páginas.	Permite renderizar páginas desde Flask y organizar la interfaz del catálogo, panel administrativo y ventas.
Bootstrap	Base visual responsive.	Acelera la construcción de una interfaz limpia, adaptable a móvil y consistente con el prototipo aprobado.
CSS	Personalización visual.	Permite implementar el Design System, colores por categoría y ajustes propios del proyecto.
SQLite	Base de datos relacional.	Es suficiente para el alcance académico y permite modelar productos, inventario, pedidos, cotizaciones y usuarios.
Git	Control de versiones.	Registra cambios, facilita ramas feature y mantiene trazabilidad del desarrollo.
GitHub	Repositorio remoto y Pull Requests.	Permite revisar cambios, trabajar en pareja y mantener dev/main según el Roadmap.
Vercel	Plataforma de despliegue.	Se usa como destino de publicación final indicado para la entrega del proyecto.

7.1 Librerías de soporte aprobadas por el Roadmap

El Roadmap de la Fase 2 indica preparar dependencias base para Flask. Estas librerías no cambian la arquitectura; apoyan autenticación, formularios, persistencia y estilos dentro de la solución aprobada.

Librería de soporte	Uso previsto
Flask-Login	Control de sesión para administrador y vendedores.
Flask-WTF	Formularios y validaciones de entrada.
SQLAlchemy	Modelado relacional entre Python y SQLite.
Bootstrap integration	Integración práctica de componentes visuales aprobados.

8. Arquitectura modular

La arquitectura modular es una decisión aprobada en el ADR y una condición del Roadmap. Su propósito es permitir que el sistema se desarrolle por partes sin duplicar información ni romper módulos ya construidos.

Arquitectura modular por capas

Separación de responsabilidades dentro de cada módulo



Figura 3. Arquitectura modular por capas.

8.1 Módulos principales de negocio

Módulo	Responsabilidad	Relación con otros módulos
Módulo Público	Mostrar catálogo, categorías, detalle de producto y cotizaciones.	Lee productos activos y disponibilidad pública desde la fuente única de productos e inventario.
Módulo Administrativo	Gestionar productos, inventario, usuarios, ventas, cotizaciones e historial.	Mantiene datos internos que alimentan el catálogo y operaciones.
Módulo de Vendedores	Buscar productos, crear pedidos, marcar pago y generar comprobantes.	Usa productos e inventario; al pagar actualiza stock y registra movimiento.

8.2 Capas internas por módulo

Capa	Qué contiene	Qué no debe contener
Routes	Rutas HTTP, recepción de formularios, selección de template.	Reglas complejas de negocio o consultas duplicadas.
Services	Cálculos, validaciones, reglas de mínimos, estados y coordinación entre modelos.	Estructura visual HTML.
Models	Entidades de base de datos y relaciones.	Decisiones visuales o navegación.
Templates	HTML con Jinja2, Bootstrap y datos ya preparados.	Cálculos críticos de inventario o permisos.
Static	CSS, imágenes y recursos visuales.	Reglas del negocio o datos persistentes.

8.3 Comunicación entre módulos

Los módulos no deben comunicarse copiando datos. La integración ocurre mediante servicios y modelos compartidos. Por ejemplo, ventas no debe tener una copia del producto; debe consultar el producto existente y registrar ítems de pedido relacionados con ese producto.

- El catálogo lee productos activos, pero no modifica inventario.
- Administración crea y actualiza productos, estados y existencias internas.
- Ventas crea pedidos y actualiza inventario únicamente cuando el pedido queda pagado.
- Cotizaciones reutilizan productos y pueden convertirse en pedido sin duplicar información base.

9. Estructura completa del proyecto

La siguiente estructura es la guía técnica para crear el repositorio durante la Fase 2. Está alineada con la decisión de arquitectura modular por carpetas y con la necesidad del Roadmap de preparar templates, static, models, services y routes por módulo.

Carpeta / archivo	Responsabilidad
/	Raíz del repositorio con archivos de configuración, README y punto de entrada.
run.py	Archivo de entrada local para iniciar la aplicación Flask.
config.py	Configuración central del proyecto, conexión a SQLite y variables de entorno necesarias.
requirements.txt	Lista de dependencias aprobadas para instalar el entorno.
README.md	Instrucciones de ejecución, descripción del sistema y enlace final de deploy.
app/	Paquete principal de la aplicación Flask.
app/__init__.py	Crea la aplicación, registra Blueprints, inicializa extensiones y configuración.
app/models/	Modelos de base de datos: productos, inventario, usuarios, cotizaciones y pedidos.
app/services/	Servicios de negocio compartidos: catálogo, inventario, ventas y cotizaciones.
app/routes/ o app/blueprints/	Rutas agrupadas por módulo funcional.
app/templates/	Plantillas Jinja2 organizadas por área: público, admin, ventas, cotizaciones y parciales.
app/static/	CSS, imágenes y recursos visuales del proyecto.
app/forms/	Formularios internos para productos, inventario, login, cotizaciones y pedidos cuando aplique.
instance/	Ubicación recomendada para la base de datos SQLite local y archivos no versionados.
migrations/ o database/	Espacio para scripts o archivos de inicialización de base de datos si se requiere en la Fase 2.
tests/ o checklist/	Pruebas manuales o listas de validación de flujos principales para la fase de pruebas.
.gitignore	Evita subir archivos temporales, entorno virtual, base local sensible y cachés.
Regla de estructura La organización exacta debe conservar una separación clara entre rutas, servicios, modelos, templates y static. No se deben mezclar módulos en un único archivo grande.	

10. Explicación de cada archivo importante

Los archivos importantes se describen a nivel de responsabilidad. Este documento no incluye código fuente, pero sí define qué debe existir y qué función cumple cada pieza dentro de la arquitectura.

Archivo o grupo	Función	Criterio técnico
run.py	Inicia la aplicación en entorno local.	Debe ser simple y delegar la creación real de la app al paquete app.
config.py	Centraliza configuración.	Evita valores dispersos dentro de rutas o servicios.
app/__init__.py	Crea la app Flask y registra módulos.	Debe permitir que cada Blueprint se conecte sin mezclar responsabilidades.
models/product.py	Representa productos, categorías y estado.	Mantiene el código único y relación con catálogo, inventario y ventas.
models/inventory.py	Representa existencias y movimientos.	Permite historial y trazabilidad de ajustes.
models/order.py	Representa pedidos, ítems y estado de pago.	Debe soportar descuento de inventario solo al quedar pagado.
models/quote.py	Representa cotizaciones e ítems.	Debe permitir resumen visual y posible conversión a pedido.
models/user.py	Representa administrador y vendedores.	Debe diferenciar permisos sin exponer datos sensibles al público.
services/catalog_service.py	Prepara productos visibles y disponibilidad pública.	No debe revelar stock exacto.
services/inventory_service.py	Gestiona ajustes y movimientos.	Debe registrar usuario, fecha y motivo.
services/sales_service.py	Coordina pedidos, pago y descuento de stock.	Debe proteger la regla de inventario al pagar.
services/quote_service.py	Calcula y organiza cotizaciones.	Debe reutilizar productos existentes.
templates/base.html	Base visual del sistema.	Reduce repetición y mantiene consistencia.
static/css/	Estilos propios.	Implementa Design System y colores de categorías.

11. Organización de Templates

Los templates deben organizarse por módulo para mantener claridad visual y evitar que una pantalla de ventas termine mezclada con el catálogo público o con administración.

Ruta de templates	Contenido esperado
templates/base.html	Estructura global: head, Bootstrap, CSS, bloques, mensajes y layout base.
templates/partials/	Navbar, footer, tarjetas reutilizables, alertas y componentes comunes.
templates/public/	Home, catálogo, filtros, detalle de producto y cotización pública.
templates/admin/	Dashboard, listados, formularios de productos, inventario, usuarios y detalles internos.
templates/sales/	Buscador de vendedor, nuevo pedido, confirmación de pago y comprobante.
templates/quotes/	Vista de cotización, resumen y documento visual.
templates/errors/	Páginas simples de error para rutas no encontradas o acceso no autorizado.

11.1 Buenas prácticas para templates

- Usar herencia desde la plantilla base para evitar duplicación de HTML.
- Mantener parciales pequeños y reutilizables.
- No colocar reglas de inventario, mínimos de venta o permisos complejos dentro del HTML.
- Usar textos claros para explicar presentación comercial, precio y disponibilidad.
- Mantener consistencia visual entre módulos mediante Bootstrap y CSS propio.

12. Organización de Static

La carpeta static concentra los recursos visuales que no son templates: CSS, imágenes, íconos y elementos gráficos necesarios para la interfaz. Su organización debe respetar el Design System único definido para el proyecto.

Ruta static	Responsabilidad
static/css/base.css	Estilos generales del sistema, tipografía, espaciados y variables visuales.
static/css/public.css	Ajustes del catálogo público, tarjetas de producto y filtros.
static/css/admin.css	Ajustes del panel administrativo, tablas, formularios y dashboard.
static/css/sales.css	Ajustes del flujo de vendedor, buscador, pedido y comprobante.
static/img/products/	Imágenes de productos usadas por catálogo, administración y ventas.
static/img/ui/	Logos, banners, iconos o recursos del Design System.
static/vendor/ o CDN aprobado	Recursos de Bootstrap cuando se decida cargarlos de forma local o externa según la configuración del proyecto.
Regla visual Las categorías tendrán identidad visual: cosméticos, juguetes y flores deben poder distinguirse rápidamente sin alterar la lógica del negocio.	

13. Organización de la Base de Datos

La base de datos se organiza en SQLite y se modela de forma relacional. El objetivo es mantener una sola fuente de verdad para productos y conectar esa información con inventario, cotizaciones y ventas.

Organización conceptual de la base de datos

Relaciones principales sin agregar reglas fuera del alcance

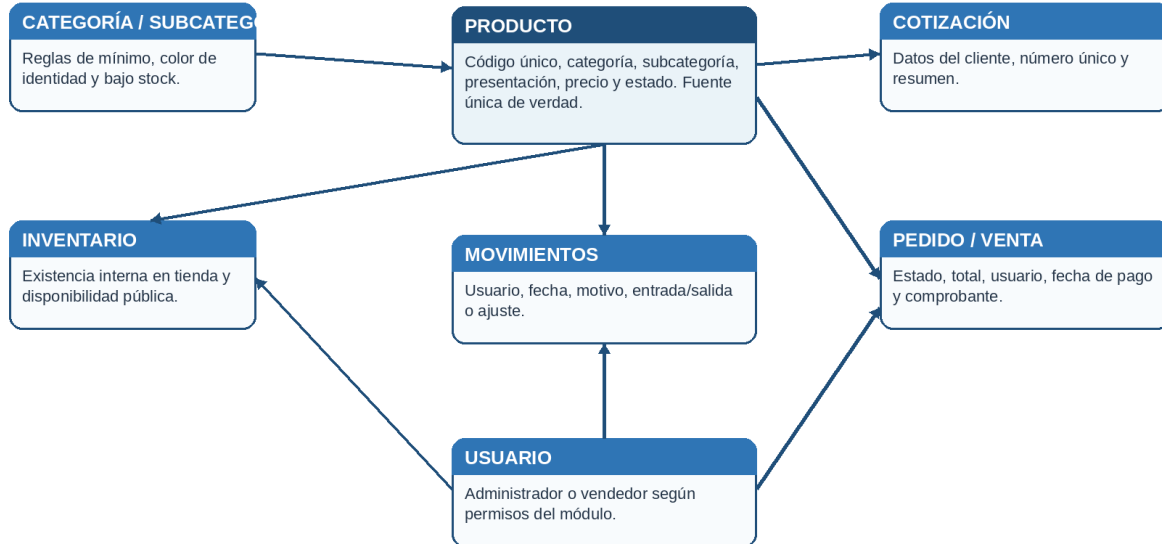


Figura 4. Organización conceptual de la base de datos.

13.1 Entidades principales

Entidad	Propósito	Reglas asociadas
Product	Información central del producto.	Código único, estado activo/inactivo/archivado, categoría, presentación y precio.
Category / Subcategory	Clasificación y configuración comercial.	Colores de identidad y umbral de bajo stock por subcategoría.
Inventory	Existencia interna en tienda.	El público no ve cantidades exactas.
InventoryMovement	Historial de entradas, salidas y ajustes.	Debe registrar usuario, fecha y motivo.
Quote / QuoteItem	Cotización y detalle de productos.	Calcula subtotales y totales sin manejar pagos en línea.
Order / OrderItem	Pedido o venta y sus productos.	Descuenta inventario solo cuando el estado queda pagado.
User	Administrador o vendedor.	Controla permisos y trazabilidad de acciones internas.

13.2 Estados relevantes

Tipo de estado	Valores aprobados	Uso técnico
Producto	Activo, Inactivo, Archivado	Controla visibilidad pública y conservación de historial.
Disponibilidad pública	Disponible, Baja disponibilidad, Agotado	Se muestra al cliente sin revelar stock exacto.
Pedido	Borrador, Pendiente de pago, Pagado, Cancelado	Controla flujo de venta e inventario.

13.3 Criterios de integridad

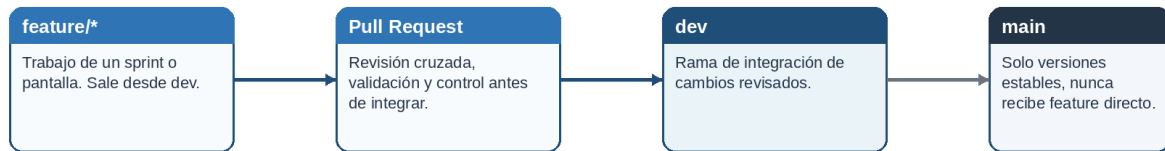
- Un producto no se duplica por aparecer en catálogo, inventario, cotización o venta.
- Los pedidos e ítems conservan relación con el producto usado en la operación.
- El descuento de inventario ocurre una sola vez al confirmar pago.
- Los ajustes manuales de inventario dejan historial interno.
- La base de datos no maneja impuestos porque el proyecto no incluye facturación fiscal.

14. Git Flow del proyecto

El flujo de Git está definido por el Roadmap y debe cumplirse durante todo el desarrollo. La regla operativa central es que las ramas feature no se integran directamente a main. Todo cambio de trabajo pasa por Pull Request hacia dev.

Git Flow aprobado

Ramas feature, Pull Request y rama dev como integración



Regla diaria obligatoria: feature/* -> Pull Request -> dev. Nunca feature/* -> main

La rama main se conserva como referencia estable para entregas controladas, de acuerdo con el Roadmap.

Figura 5. Git Flow aprobado para el proyecto.

Rama	Uso	Regla
main	Versión estable y entregable.	No recibe ramas feature directamente.
dev	Integración de cambios revisados.	Recibe Pull Requests desde feature/*.
feature/<módulo>	Trabajo de un sprint, pantalla o módulo.	Debe salir de dev y volver a dev por Pull Request.

14.1 Flujo obligatorio

feature/* -> Pull Request -> dev. Nunca feature/* -> main.

- Cada sprint debe tener una rama feature clara.
- Los commits deben ser descriptivos y en inglés, como indica el Roadmap.
- La revisión cruzada entre integrantes es obligatoria antes del merge.
- La rama main se mantiene como referencia de entrega estable, no como rama de trabajo diario.

15. Flujo de desarrollo

El desarrollo se ejecuta siguiendo el orden del Roadmap. Cada chat y cada rama deben enfocarse en un módulo o pantalla para evitar mezclar responsabilidades.

Paso	Acción	Resultado esperado
1	Revisar Documento 0, ADR y Roadmap.	Se confirma alcance y reglas antes de trabajar.
2	Crear o actualizar rama feature desde dev.	El trabajo queda aislado por módulo.
3	Construir rutas, servicios, modelos, templates y static del módulo.	El módulo cumple su responsabilidad sin afectar otros.
4	Probar flujo localmente.	Se valida navegación, formularios, cálculo y permisos aplicables.
5	Hacer commits descriptivos en inglés.	El historial se mantiene claro.
6	Abrir Pull Request hacia dev.	El cambio se revisa antes de integrarse.
7	Actualizar estado en chat MASTER si corresponde.	La documentación y el avance quedan sincronizados.

15.1 Fases del Roadmap que gobiernan el desarrollo

Fase	Nombre	Resultado técnico
1	Análisis y diseño	Sistema entendido y documentado. Completada.
2	Configuración del proyecto	Repositorio, estructura Flask, dependencias y base técnica.

3	Módulo público	Home, catálogo, detalle y cotización.
4	Panel administrativo	Productos, inventario, usuarios y dashboard.
5	Sistema de ventas	Pedidos, pagos y comprobantes.
6	Integración	Catálogo, inventario, cotizaciones y ventas conectados.
7	Pruebas y optimización	Flujos validados y experiencia corregida.
8	Deploy y entrega	Proyecto publicado, documentado y listo para presentación.

16. Convenciones de nombres

Las convenciones permiten que el proyecto sea fácil de leer, explicar y mantener. Deben aplicarse desde la Fase 2 para evitar reorganizaciones innecesarias.

Elemento	Convención	Ejemplo
Branches	feature/<nombre-corto-en-inglés>	feature/catalog, feature/admin-products, feature/sales-payment
Commits	Mensaje descriptivo en inglés	Add product listing filters
Archivos Python	lower_snake_case	catalog_service.py, inventory_service.py
Clases de modelos	PascalCase	Product, InventoryMovement, Order
Templates	Nombres claros por módulo	public/catalog.html, sales/order_detail.html
CSS	Nombre por área o responsabilidad	public.css, admin.css, sales.css
Tablas de base de datos	Nombres consistentes y legibles	products, orders, order_items
Códigos de producto	Se conserva el código único definido por la empresa	No se cambia ni se genera uno distinto si ya existe.

16.1 Convenciones de contenido visible

- Usar términos consistentes: producto, cotización, pedido, pagado, cancelado, inventario en tienda.
- No usar “stock exacto” en pantallas públicas; solo disponibilidad pública.
- Mostrar presentación comercial de forma clara: caja, display, unidad comercial o ramo según categoría.
- Mantener el lenguaje de la interfaz simple para clientes mayoristas y vendedores.

17. Buenas prácticas

Las buenas prácticas no agregan nuevas reglas de negocio; ayudan a implementar correctamente las reglas ya aprobadas.

Área	Buena práctica	Motivo
Arquitectura	Separar rutas, servicios, modelos y templates.	Evita archivos grandes y facilita trabajo por módulos.
Datos	No duplicar productos para catálogo, inventario o ventas.	Conserva una fuente única de verdad.
Inventario	Usar transacciones lógicas al marcar pago.	Evita descontar inventario antes de tiempo o dos veces.
Productos	No eliminar físicamente productos.	Conserva historial de ventas y cotizaciones.
Seguridad	Restringir paneles internos por rol.	Protege información sensible del negocio.
Templates	Mantener lógica compleja fuera de HTML.	Mejora mantenimiento y evita errores visuales.
Git	Revisar Pull Requests antes de integrar.	Reduce errores y cumple el flujo del Roadmap.
UI	Aplicar Design System y Bootstrap consistentemente.	Mantiene apariencia profesional y responsive.
Documentación	Actualizar el chat MASTER cuando haya decisiones relevantes.	Mantiene trazabilidad y coherencia entre documentos.

17.1 Criterios de Definition of Done global

- La estructura modular está creada y entendible.
- Rutas y servicios funcionan para el flujo del módulo.
- Validaciones básicas se cumplen según reglas aprobadas.
- La interfaz es clara y responsive.
- La integración con módulos dependientes no rompe flujos previos.
- Existe commit, Pull Request y revisión antes de integrar a dev.

18. Cumplimiento de los lineamientos del profesor

La siguiente tabla compara los lineamientos explícitos conocidos para el proyecto con la forma en que se implementan en la arquitectura. No se agregan requisitos externos no documentados.

Requisito del profesor	Cómo será implementado
Aplicación web funcional	Flask entrega páginas HTML con catálogo, administración, ventas y cotizaciones.
Uso de Flask	La aplicación se organiza con Flask, Blueprints, rutas, servicios, modelos y templates.
Uso de templates HTML	Jinja2 renderiza base.html, parciales y pantallas por módulo.
Interfaz responsive	Bootstrap y CSS propio permiten pantallas adaptables para cliente, admin y vendedor.
Base de datos	SQLite almacena productos, inventario, usuarios, cotizaciones, pedidos y movimientos.
Arquitectura modular	El proyecto se divide en módulos público, administrativo, vendedores y submódulos técnicos.
CRUD o gestión de información	El panel administrativo gestiona productos, estados, inventario y usuarios.
Control de inventario	Inventario interno con historial y descuento solo cuando el pedido está pagado.
Reglas de negocio reales	Mínimos por categoría, presentación comercial y estados aprobados se aplican en servicios.
Roles o accesos diferenciados	Cliente público, administrador y vendedor tienen vistas y permisos diferentes.
Git y ramas	feature/* -> Pull Request -> dev; nunca feature directo a main.
GitHub	Repositorio remoto, revisión cruzada y trazabilidad de Pull Requests.
Deploy	Publicación final en Vercel durante la Fase 8.
Documentación técnica	Documento 2 define arquitectura, carpetas, archivos, flujos, diagramas y buenas prácticas.
Presentación defendible	El documento incluye preguntas y respuestas técnicas para explicar decisiones.
Alcance controlado	No se incluyen pagos en línea, SAT, inventario de bodega ni proveedores.

19. Diagramas de arquitectura

Los siguientes diagramas resumen la arquitectura técnica. Están pensados para explicar el sistema durante el desarrollo y durante la presentación.

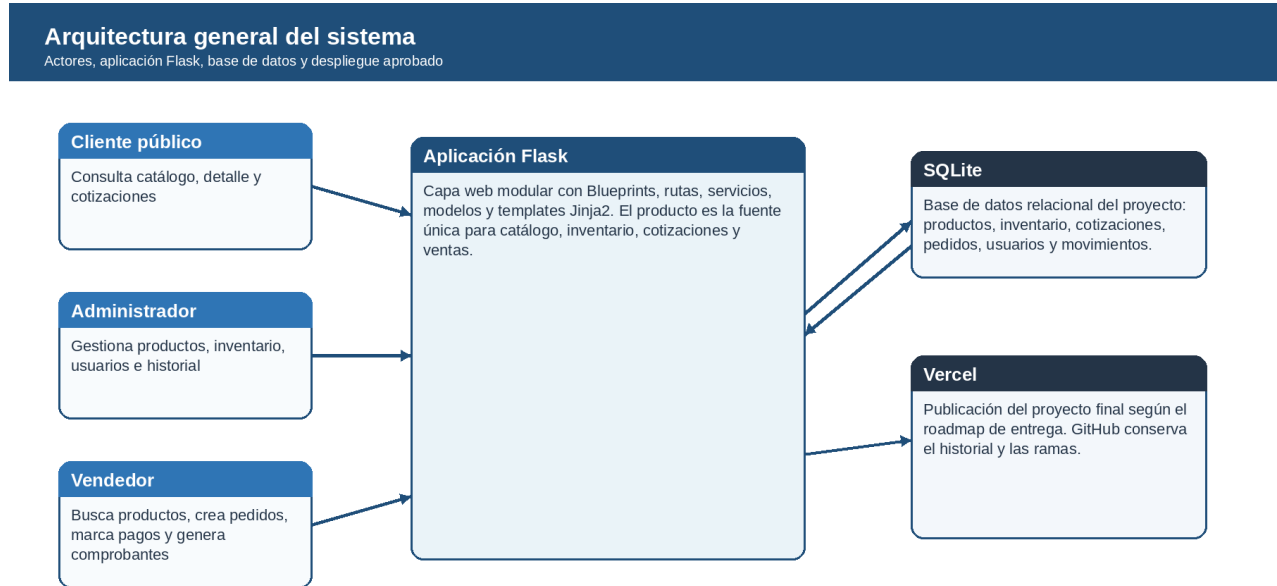


Figura 6. Vista general de actores, Flask, SQLite y Vercel.



Figura 7. Capas internas de la arquitectura modular.

Organización conceptual de la base de datos

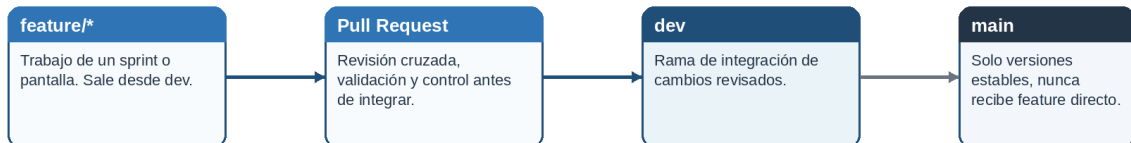
Relaciones principales sin agregar reglas fuera del alcance



Figura 8. Modelo conceptual de datos y fuente única del producto.

Git Flow aprobado

Ramas feature, Pull Request y rama dev como integración



Regla diaria obligatoria: feature/* -> Pull Request -> dev. Nunca feature/* -> main

La rama main se conserva como referencia estable para entregas controladas, de acuerdo con el Roadmap.

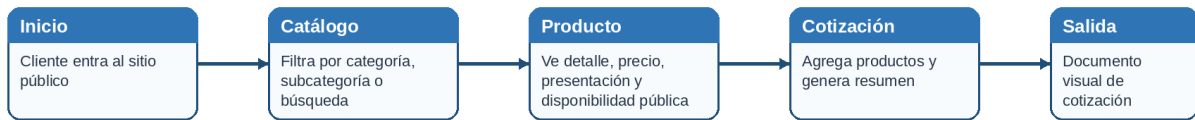
Figura 9. Flujo de ramas Git del proyecto.

20. Diagramas de flujo

Los diagramas de flujo muestran cómo se ejecutan las operaciones principales del sistema sin mezclar reglas de negocio ni responsabilidades técnicas.

Flujo público: catálogo y cotización

Navegación del cliente mayorista sin pagos en línea



Reglas públicas

Solo productos activos. El cliente ve Disponible, Baja disponibilidad o Agotado, pero nunca cantidades exactas de inventario.

Figura 10. Flujo público de navegación, producto y cotización.

Flujo de ventas e inventario

Pedido, pago confirmado y descuento de inventario en tienda

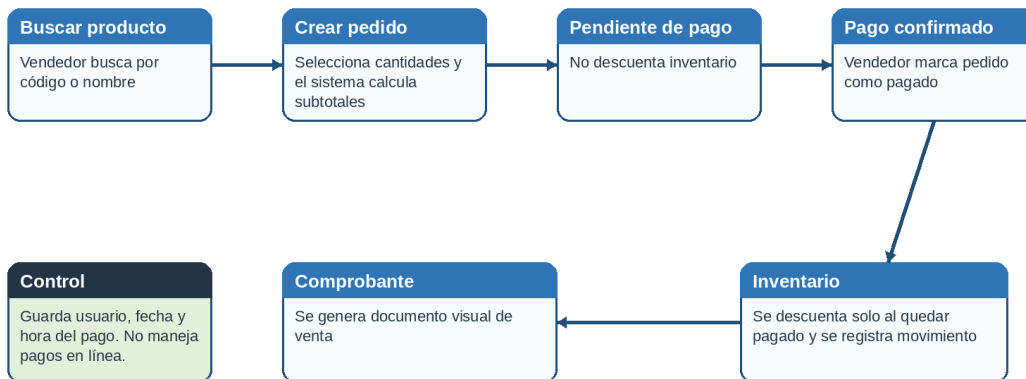
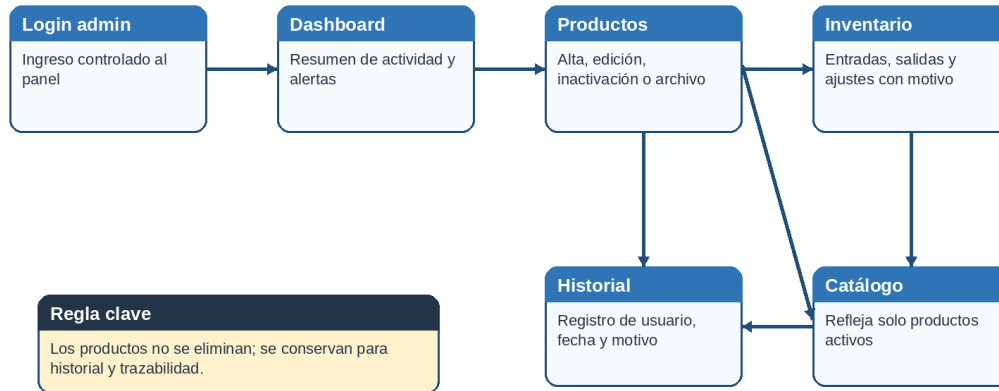


Figura 11. Flujo de venta, pago y descuento de inventario.

Flujo administrativo

Gestión interna de productos e inventario

*Figura 12. Flujo administrativo de productos e inventario.***20.1 Puntos críticos de los flujos**

- El catálogo público nunca expone cantidades exactas de inventario.
- La cotización no descuenta inventario por sí misma.
- El pedido pendiente de pago no modifica stock.
- El pago confirmado registra usuario, fecha y hora antes de descontar inventario.
- Los ajustes administrativos de inventario requieren registro de motivo para conservar trazabilidad.

21. Preguntas para la presentación

Esta sección prepara respuestas técnicas claras para preguntas probables durante la presentación. Las respuestas están alineadas con el alcance y las decisiones aprobadas.

Pregunta del profesor	Respuesta técnica clara
¿Por qué eligieron Flask?	Porque permite construir una aplicación web modular, ligera y fácil de explicar. Con Blueprints se separan catálogo, administración y ventas sin cambiar el alcance.
¿Qué problema resuelve el sistema?	Centraliza producto, catálogo, inventario, cotizaciones y ventas para reducir errores manuales y mantener trazabilidad.
¿Por qué el producto es la fuente única de verdad?	Porque evita crear productos duplicados para cada módulo. El mismo producto alimenta catálogo, inventario, cotización y venta.
¿Cuándo se descuenta inventario?	Solo cuando el pedido está marcado como pagado. Un pedido borrador o pendiente de pago no modifica existencias.
¿Por qué el cliente no ve el stock exacto?	Porque el stock exacto es información operativa interna. El público solo ve estados como Disponible, Baja disponibilidad o Agotado.
¿Por qué los productos no se eliminan?	Porque ventas y cotizaciones pasadas necesitan historial. Por eso se usan estados: activo, inactivo o archivado.
¿Cómo se organiza Flask?	Con una app principal, Blueprints por módulo, modelos para datos, servicios para reglas, templates Jinja2 para vistas y static para estilos.
¿Qué hace Jinja2?	Convierte datos enviados por Flask en HTML. Permite reutilizar una base visual y crear pantallas por módulo.
¿Por qué SQLite?	Porque el alcance del proyecto requiere una base relacional simple, suficiente para productos, inventario, pedidos, cotizaciones y usuarios.
¿Cómo evitan mezclar lógica con HTML?	Las reglas de negocio se colocan en servicios y modelos. Las plantillas solo presentan datos y mensajes visuales.
¿Qué diferencia hay entre administrador y vendedor?	El administrador gestiona productos, inventario y usuarios. El vendedor busca productos, crea pedidos, marca pagos y genera comprobantes.
¿Cómo se controlan las reglas de mínimos de venta?	Se aplican en servicios antes de aceptar cotizaciones o pedidos, respetando cosméticos, juguetes y flores.
¿Qué pasa con una cotización?	Se genera como documento o resumen visual. Puede servir como base de pedido sin duplicar productos.
¿Por qué usan Bootstrap?	Porque acelera una interfaz responsive y profesional, coherente con el prototipo HTML + Bootstrap aprobado.
¿Cómo trabajan en equipo con Git?	Cada cambio se hace en feature/*, se revisa por Pull Request y se integra a dev. Nunca se manda feature directo a main.
¿Qué papel tiene GitHub?	Guarda el repositorio, permite revisar Pull Requests y conserva historial de commits.
¿Qué se despliega en Vercel?	La aplicación final documentada y lista para evaluación, según la Fase 8 del Roadmap.
¿Qué módulos no están incluidos?	No se incluye SAT, pagos en línea, inventario de bodega, proveedores ni control de importaciones.
¿Cómo prueban el sistema?	Validando navegación pública, productos, inventario, permisos, ventas, pagos, comprobantes y responsividad.
¿Cómo garantizan que no se rompa la arquitectura?	Cualquier cambio importante se revisa contra el ADR, el Roadmap y el chat MASTER antes de integrarse.

Cierre técnico del Documento 2

La arquitectura técnica definida en este documento deja preparado el camino para construir el sistema en el orden del Roadmap: configuración Flask, módulo público, panel administrativo, ventas, integración, pruebas y despliegue. El documento mantiene las decisiones aprobadas y convierte el contexto del proyecto en una guía concreta para el desarrollo.

Regla final

Durante la implementación, ningún módulo debe cambiar decisiones aprobadas. Si surge una decisión nueva, se registra primero en el chat MASTER y luego se actualiza el ADR y la documentación correspondiente.